Динамическое программирование

Общие сведения, уравнение Беллмана, наибольшая общая подпоследовательность и задача о размене в неканонической системе номиналов

Общие сведения

Что есть «динамическое программирование» (ДМ)

Динамическим программированием называется способ решения алгоритмически разрешимых задач путём разбиения их на более простые подзадачи и объединения их решений.

Построение алгоритма на базе метода динамического программирования предполагает следующие шаги:

1. описание структуры решения в терминах меньших задач (поиск оптимальной подструктуры);
2. вычисление решений для меньших задач;
3. опираясь на анализ из пункта 1, объединить решения подзадач в решение общей задачи.

Принцип оптимальности Ричарда Беллмана
Richard Ernest Bellman

Каков бы ни был путь системы в текущее состояние, следующий шаг управления должен быть выбран таким образом, чтобы выигрыш на данном шаге плюс оптимальный выигрыш на всех последующий шагах оказался оптимальным.

ξ_k – состояние системы на шаге k

u_k – управление системы на шаге k

$f_k(\xi_{k-1}, u_k)$ – выигрыш на шаге k

W_k – значение полного выигрыша на шаге k

$$W_k = \max_{u_k}\{f_k(\xi_{k-1}, u_k) + W_{k+1}(u_k)\}$$ Функция выиигрыша

$$G_k = \min_{u_k}\{f_k(\xi_{k-1}, u_k) + G_{k+1}(u_k)\}$$ Функция штрафа

Нисходящее и восходящее ДМ

Нисходящее ДМ предполагает разделение задачи на подзадачи, которые, в свою очередь, также разбиваются на меньшие подзадачи и т.д. до тех пор пока решение задачи не становится тривиальным.

Восходящее ДМ предполагает предварительное вычисление всех необходимых тривиальных подзадач и последующее конструирование решения.

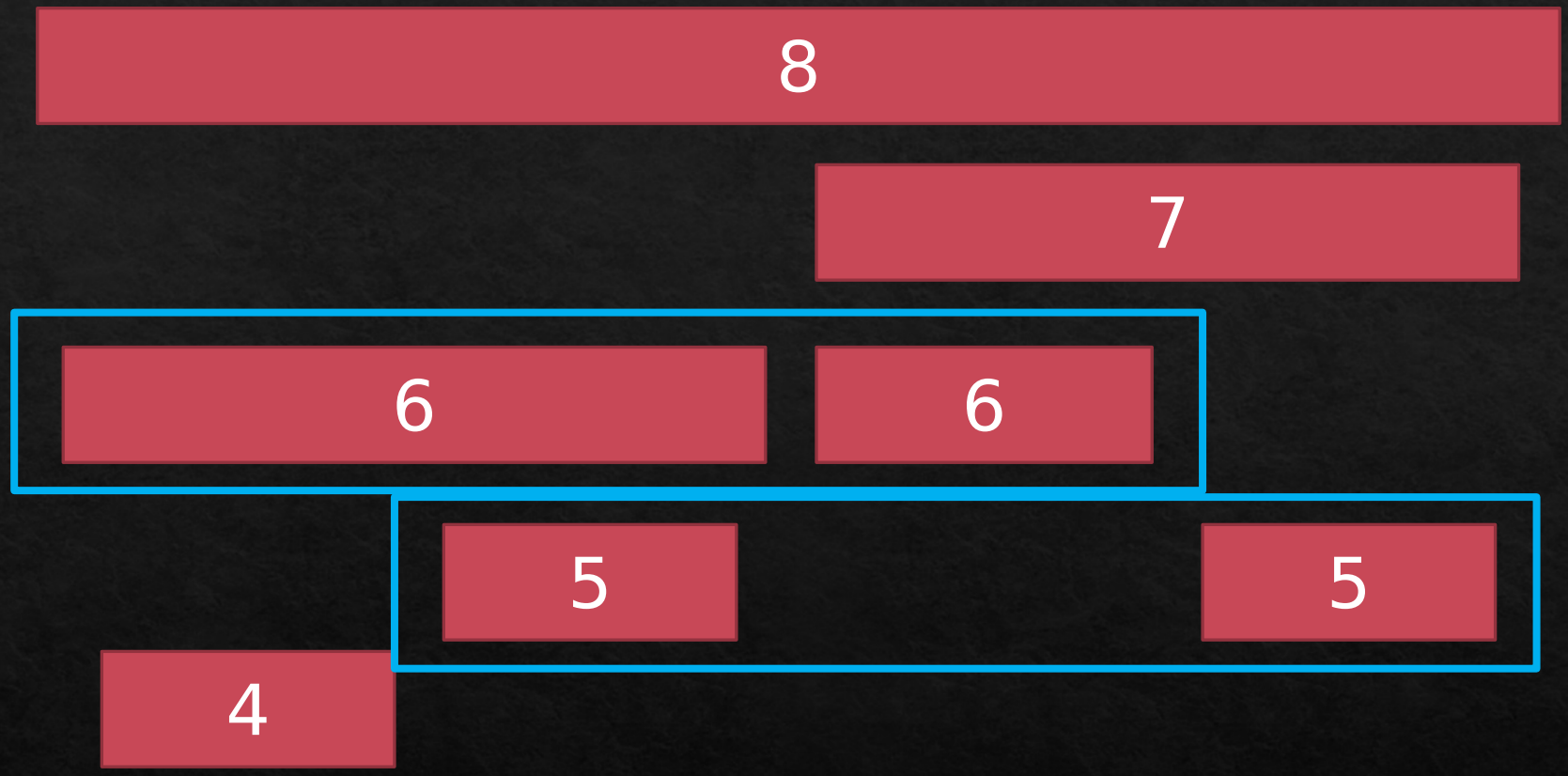
Перекрывающиеся подзадачи и «мемоизация»

Если две подзадачи одинаковы или их решение базируется на некотором общем вычислении, то такие подзадачи мы будем называть перекрывающимися. Если каждую из таких задач решать независимым образом, то мы выполним одно и то же вычисление дважды и более раз. Чтобы этого избежать мы можем либо заранее вычислить эти значения при восходящем динамическом программировании, либо, запомнив решение подзадачи при первом вычислении, общаться к нему далее при нисходящем динамическом программировании. Последний приём носит название мемоизация.

Числа Фибоначчи

Использование мемоизации

Перекрывающиеся подзадачи



Нисходящее ДМ с мемоизацией

```
1  #include <iostream>
2
3  unsigned fibonacci_dn(unsigned fib_number, unsigned const * const memoization) {
4      if (0 != memoization[fib_number])
5          return memoization[fib_number];
6      return
7          fibonacci_dn(fib_number - 1, memoization) +
8          fibonacci_dn(fib_number - 2, memoization);
9  }
10
11 using namespace std;
12
13 int main() {
14     unsigned fib_number;
15     cin >> fib_number;
16     auto const number_memoization = new unsigned[fib_number + 1] {0};
17     number_memoization[1] = 1;
18     number_memoization[2] = 1;
19
20     cout << fibonacci_dn(fib_number, number_memoization) << endl;
21     delete[] number_memoization;
22
23     return 0;
24 }
```

если задача решалась ранее, то мы используем запомненное значение

рекурсивный вызов

тривиальные задачи, решение которых мы знаем

Восходящее ДМ

```cpp
#include <iostream>

using namespace std;

unsigned fibonacci_up(unsigned fib_number) {
    auto const numbers = new unsigned[fib_number + 1] {0};
    numbers[1] = 1;
    numbers[2] = 1;

    for (unsigned curr_num_idx = 3; curr_num_idx != fib_number + 1; ++curr_num_idx)
        numbers[curr_num_idx] = numbers[curr_num_idx - 2] + numbers[curr_num_idx - 1];

    auto result = numbers[fib_number];
    delete[] numbers;
    return result;
}

int main() {
    unsigned fib_number;
    cin >> fib_number;
    cout << fibonacci_up(fib_number) << endl;

    return 0;
}
```

тривиальные задачи, решение которых мы знаем

вычисляем все подзадачи, используя знания о предыдущих подзадачах

Наибольшая общая подпоследовательность

Формулировка

Даны две последовательности. Необходимо найти какова наибольшая длина подпоследовательностей, состоящих из одинаковых элементов. Подпоследовательность – это набор элементов с возрастающими (но не обязательно строго последовательными) индексами.

Последовательность {1, 2, 3, 4, 5} содержит подпоследовательности {1,2,3}, {2,5,4}, {1,3,5} и т.д.

Пусть последовательность A = {1, 2, 3, 4, 5}
Последовательность B = {2, 3, 6, 5}
Тогда {2, 5} – подпоследовательность длины 2, содержащаяся в обеих последовательностях.

Алгоритм



Предположим, что мы сделали все оптимальные выборы и решаем вопрос о последнем оптимальном W_n. Тогда нам уже не нужно рассматривать дальнейших выборов, так как это последний шаг «управления».

Если элементы совпадают ($A_n == B_n$), то оптимальное решение есть уже сделанные оптимальные шаги и ещё один символ. Если не совпадают, то нужно выбрать один из двух предыдущих оптимальных выборов (максимизация предыдущего управления).

$$W(n_a, n_b) = \begin{cases} W(n_a - 1, n_b - 1) + 1 \\ max\big(W(n_a, n_b - 1), W(n_a - 1, n_b)\big) \end{cases}$$

Нисходящее ДМ

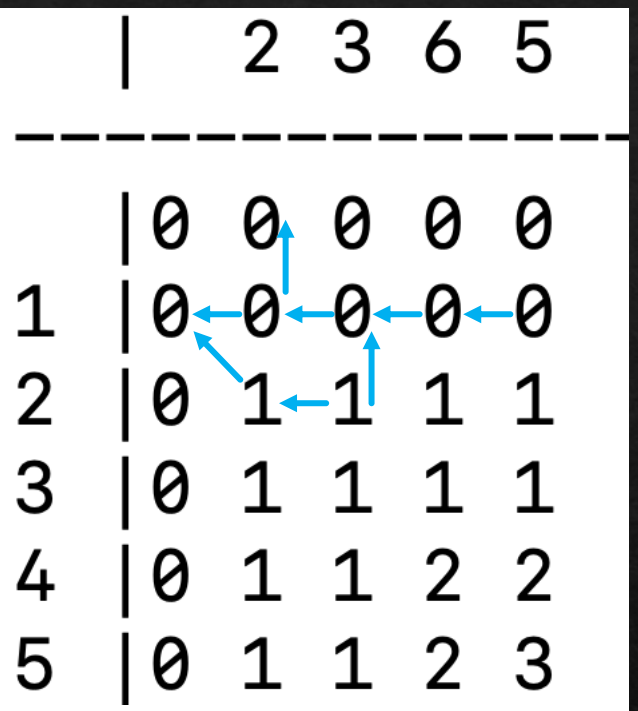
```
4  unsigned greatest_common_subseq_rec(
5      int const *seqA, unsigned const sizeA,
6      int const *seqB, unsigned const sizeB,
7      int **memoization) {
8
9      if (0 == sizeA || 0 == sizeB) {
10         if (0 == memoization[sizeA][sizeB]) memoization[sizeA][sizeB] = 1;
11         return memoization[sizeA][sizeB];
12     }
13
14     if (seqA[sizeA - 1] == seqB[sizeB - 1]) {
15         if (0 == memoization[sizeA - 1][sizeB - 1]) memoization[sizeA - 1][sizeB - 1] =
16                 greatest_common_subseq_rec(seqA, sizeA - 1, seqB, sizeB - 1, memoization);
17         return memoization[sizeA - 1][sizeB - 1] + 1;
18     }
19
20     if (0 == memoization[sizeA - 1][sizeB])
21         memoization[sizeA - 1][sizeB] =
22             greatest_common_subseq_rec(seqA, sizeA - 1, seqB, sizeB, memoization);
23
24     if (0 == memoization[sizeA][sizeB - 1])
25         memoization[sizeA][sizeB - 1] =
26             greatest_common_subseq_rec(seqA, sizeA, seqB, sizeB - 1, memoization);
27
28     return std::max(memoization[sizeA - 1][sizeB], memoization[sizeA][sizeB - 1]);
29 }
```

```
5 4
1 2 3 4 5
2 3 6 5
3
```

		2	3	6	5
	0	1	1	1	0
1	1	1	1	1	0
2	0	2	2	2	0
3	1	2	3	3	0
4	1	2	3	3	0
5	0	0	0	0	0

Восходящее ДМ

```
11  void greates_common_subseq_loop(
12      int const *seqA, unsigned const sizeA,
13      int const *seqB, unsigned const sizeB,
14      int **memoization) {
15
16      for (unsigned subsizeA = 0; subsizeA != sizeA + 1; ++subsizeA)
17          memoization[subsizeA][0] = 0;
18
19      for (unsigned subsizeB = 0; subsizeB != sizeB + 1; ++subsizeB)
20          memoization[0][subsizeB] = 0;
21
22      for (unsigned subsizeA = 1; subsizeA != sizeA + 1; ++subsizeA)
23          for (unsigned subsizeB = 1; subsizeB != sizeB + 1; ++subsizeB) {
24              if (seqA[subsizeA] == seqB[subsizeB])
25                  memoization[subsizeA][subsizeB] = memoization[subsizeA - 1][subsizeB - 1] + 1;
26              else
27                  memoization[subsizeA][subsizeB] =
28                      std::max(memoization[subsizeA - 1][subsizeB], memoization[subsizeA][subsizeB - 1]);
29          }
30  }
```



Задача о размене

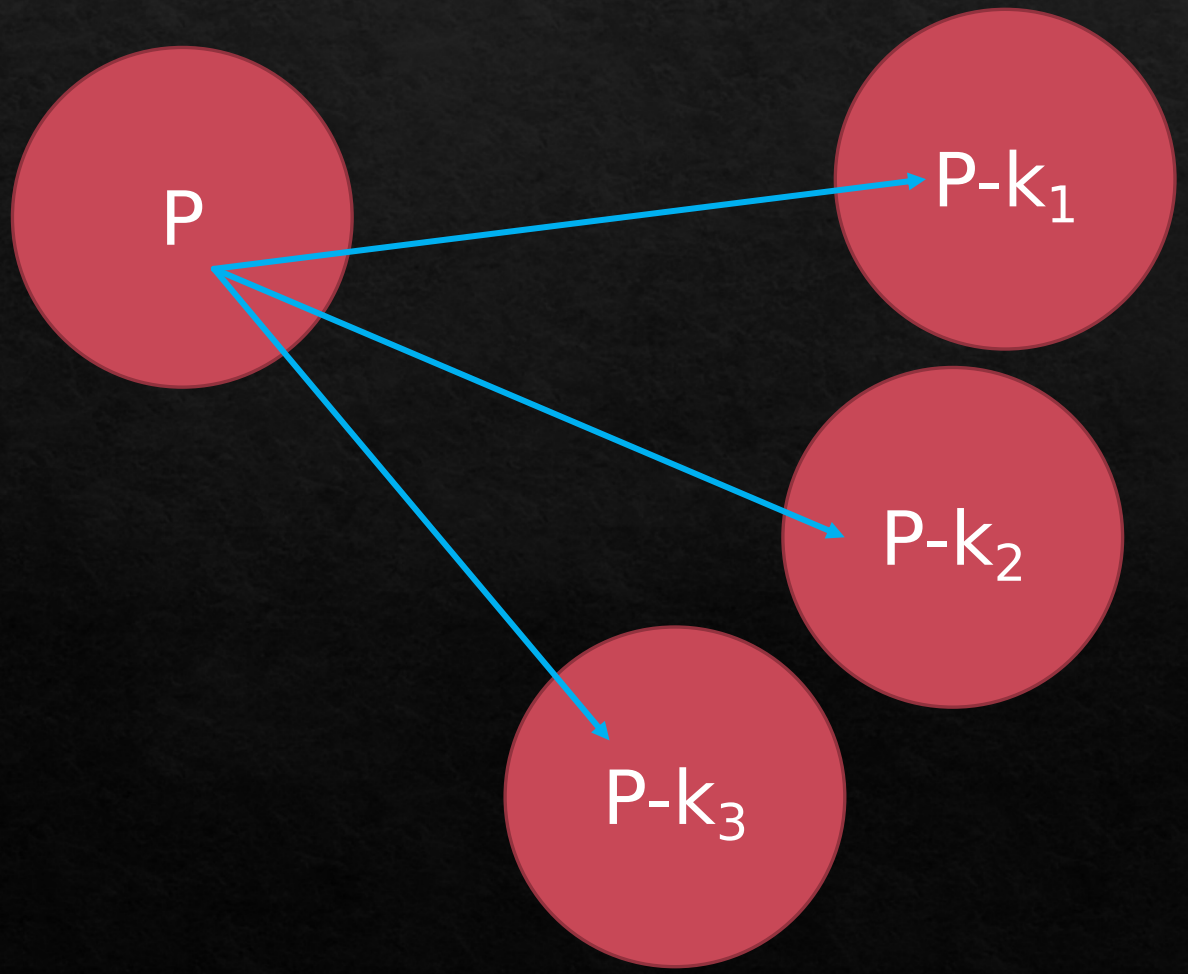
Формулировка

Дан набор монет с различным номиналом, например 1, 5, 25, 50, 100, нужно набрать некоторую заданную сумму P с использованием наименьшего числа монет. Количество монет каждого номинала не ограничено.

Мы знаем, что эта задача разрешима жадным алгоритмом для неканонических систем. Динамическое программированием поможет нам разрешить эту задачу для неканонических систем.

Построение оптимального решения

Если нам известно для некоторого ***P*** оптимальное решение, то убрав из него монету некоторого номинала k_i, мы получим оптимальное решение для $P - k_i$.



Обратное утверждение.
Если для ***P*** существует оптимальное решение, то существует такой набор монет с меньшим общим значением, что добавив к нему одну монету, мы получим оптимальное решение для ***P***.

Нисходящее ДМ

```
12  void least_coin_up(unsigned const *coins, unsigned const coins_size, unsigned *optimals, unsigned const P) {
13      optimals[0] = 0;
14      for (unsigned sum = 1; sum != P + 1; ++sum) {
15          unsigned local_min = std::numeric_limits<unsigned>::max();
16          for (unsigned coin_idx = 0; coin_idx != coins_size; ++coin_idx)
17              if (sum >= coins[coin_idx])
18                  local_min = std::min(optimals[sum - coins[coin_idx]] + 1, local_min);
19          optimals[sum] = local_min;
20      }
21  }
```